Document number	P3059R0
Date	2023-11-30
Audience	LEWG, SG9 (Ranges)
Reply-to	Hewill Kang <hewillk@gmail.com>

Making user-defined constructors of view iterators/sentinels private

Abstract

The exposure of user-defined constructors for iterators/sentinels in `<ranges>` currently does not follow consistent rules, which is reflected in the fact that some of them are public and some are private. This paper disables their visibility to comply with best practices.

Revision history

R0

Initial revision.

Discussion

What's the issue?

Currently in `<ranges>`, to access the *`meow_view`*'s member variables, *`meow_view::iterator/sentinel`* saves a pointer pointing to *`meow_view`* and provides a constructor that accepts a reference to it for initializing the pointer.

However, some of these user-defined constructors are declared as `public`, which allows users to construct them by passing in a specific view base:

```
auto base = std::views::iota(0);
auto filter = base | std::views::filter([](int) { return true; });
auto begin = decltype(filter.begin())(filter, base.begin()); // ok
auto end   = decltype(filter.end()  )(filter);               // ok
```

The author believes that we should prohibit providing these constructors to users. As the example above shows, this doesn't make much sense and provides no observable value.

It is worth noting that the above example fails to compile in libstdc++, because in libstdc++ the constructor is implemented to accept a pointer to `filter_view`, which allows it to directly pass in `this` pointer in `begin()` to save one `addressof` call.

Although this is a bug in libstdc++ according to the current wording, it is clearly a side effect of the constructor being accidentally exposed. Implementers are absolutely allowed to construct the iterator by just passing `this` to initialize the member, which is what `chunk_view::`*`iterator`*'s user-defined constructor does.

Since both methods exist in `<ranges>`, it shows that this is implementation-related and should not be perceived by users.

The author believes that apart from the default constructor and the conversion constructor which has an obvious intent, there is no reason for constructors that are only used for implementation purposes to be exposed to the user.

Proposed change

This wording is relative to [N4958](#).

> [*Drafting note*: The proposed resolution does not touch user-defined constructors of `iota_view::`*`iterator/sentinel`* because it does make sense to expose them.]

1. Modify 26.6.6.3 [range.istream.iterator], class `basic_istream_view::`*`iterator`* synopsis, as indicated:

```
namespace std::ranges {
  template<movable Val, class CharT, class Traits>
    requires default_initializable<Val> &&
             stream-extractable<Val, CharT, Traits>
  class basic_istream_view<Val, CharT, Traits>::iterator {
  public:
    using iterator_concept = input_iterator_tag;
    using difference_type = ptrdiff_t;
```

```
    using value_type = Val;

    constexpr explicit iterator(basic_istream_view& parent) noexcept;

    […]

  private:
    basic_istream_view* parent_;                                    // exposition only

    constexpr explicit iterator(basic_istream_view& parent) noexcept;    // exposition only
  };
}
```

2. Modify 26.7.8.3 [range.filter.iterator], class `filter_view::iterator` synopsis, as indicated:

```
namespace std::ranges {
  template<input_range V, indirect_unary_predicate<iterator_t<V>> Pred>
    requires view<V> && is_object_v<Pred>
  class filter_view<V, Pred>::iterator {
  private:
    iterator_t<V> current_ = iterator_t<V>();   // exposition only
    filter_view* parent_ = nullptr;             // exposition only

    constexpr iterator(filter_view& parent, iterator_t<V> current);    // exposition only

  public:
    […]

    iterator() requires default_initializable<iterator_t<V>> = default;
    constexpr iterator(filter_view& parent, iterator_t<V> current);

    […]
  };
}
```

3. Modify 26.7.8.4 [range.filter.sentinel], class `filter_view::sentinel` synopsis, as indicated:

```
namespace std::ranges {
  template<input_range V, indirect_unary_predicate<iterator_t<V>> Pred>
    requires view<V> && is_object_v<Pred>
  class filter_view<V, Pred>::sentinel {
  private:
    sentinel_t<V> end_ = sentinel_t<V>();       // exposition only

    constexpr explicit sentinel(filter_view& parent);    // exposition only

  public:
    sentinel() = default;
    constexpr explicit sentinel(filter_view& parent);

    […]
  };
}
```

4. Modify 26.7.9.3 [range.transform.iterator], class template `transform_view::iterator` synopsis, as indicated:

```
namespace std::ranges {
  template<input_range V, move_constructible F>
    requires view<V> && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<V>> &&
             can-reference<invoke_result_t<F&, range_reference_t<V>>>
  template<bool Const>
  class transform_view<V, F>::iterator {
  private:
    […]

    constexpr iterator(Parent& parent, iterator_t<Base> current);   // exposition only

  public:
    […]

    iterator() requires default_initializable<iterator_t<Base>> = default;
    constexpr iterator(Parent& parent, iterator_t<Base> current);
    […]
  };
}
```

5. Modify 26.7.9.4 [range.transform.sentinel], class template `transform_view::sentinel` synopsis, as indicated:

```
namespace std::ranges {
  template<input_range V, move_constructible F>
    requires view<V> && is_object_v<F> &&
              regular_invocable<F&, range_reference_t<V>> &&
              can-reference<invoke_result_t<F&, range_reference_t<V>>>
  template<bool Const>
  class transform_view<V, F>::sentinel {
```

```
  private:
    […]

    constexpr explicit sentinel(sentinel_t<Base> end);     // exposition only

  public:
    sentinel() = default;
    constexpr explicit sentinel(sentinel_t<Base> end);
    […]
  };
}
```

6. Modify 26.7.10.3 [range.take.sentinel], class template `take_view::sentinel` synopsis, as indicated:

```
namespace std::ranges {
  template<view V>
  template<bool Const>
  class take_view<V>::sentinel {
  private:
    […]

    constexpr explicit sentinel(sentinel_t<Base> end);     // exposition only

  public:
    sentinel() = default;
    constexpr explicit sentinel(sentinel_t<Base> end);
    […]
  };
}
```

7. Modify 26.7.11.3 [range.take.while.sentinel], class template `take_while_view::sentinel` synopsis, as indicated:

```
namespace std::ranges {
  template<view V, class Pred>
    requires input_range<V> && is_object_v<Pred> &&
             indirect_unary_predicate<const Pred, iterator_t<V>>
  template<bool Const>
  class take_while_view<V, Pred>::sentinel {
  private:
    […]

    constexpr explicit sentinel(sentinel_t<Base> end, const Pred* pred);     // exposition only

  public:
    sentinel() = default;
    constexpr explicit sentinel(sentinel_t<Base> end, const Pred* pred);
    […]
  };
}
```

8. Modify 26.7.14.4 [range.join.sentinel], class template `join_view::sentinel` synopsis, as indicated:

```
namespace std::ranges {
  template<input_range V>
    requires view<V> && input_range<range_reference_t<V>>
  template<bool Const>
  struct join_view<V>::sentinel {
  private:
    […]

    constexpr explicit sentinel(Parent& parent);     // exposition only

  public:
    sentinel() = default;

    constexpr explicit sentinel(Parent& parent);
    […]
  };
}
```

9. Modify 26.7.16.3 [range.lazy.split.outer], class template `lazy_split_view::outer-iterator` synopsis, as indicated:

```
namespace std::ranges {
  template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
             (forward_range<V> || tiny-range<Pattern>)
  template<bool Const>
  struct lazy_split_view<V, Pattern>::outer-iterator {
  private:
    […]

    constexpr explicit outer-iterator(Parent& parent)                     // exposition only
      requires (!forward_range<Base>);
    constexpr outer-iterator(Parent& parent, iterator_t<Base> current)    // exposition only
      requires forward_range<Base>;
```

```
  public:
    […]
    constexpr explicit outer-iterator(Parent& parent)
      requires (!forward_range<Base>);
    constexpr outer-iterator(Parent& parent, iterator_t<Base> current)
      requires forward_range<Base>;
    […]
  };
}
```

10. Modify 26.7.16.5 [range.lazy.split.inner], class template `lazy_split_view::inner-iterator` synopsis, as indicated:

```
namespace std::ranges {
  template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
             (forward_range<V> || tiny-range<Pattern>)
  template<bool Const>
  struct lazy_split_view<V, Pattern>::inner-iterator {
  private:
    […]

    constexpr explicit inner-iterator(outer-iterator<Const> i);     // exposition only

  public:
    […]

    inner-iterator() = default;
    constexpr explicit inner-iterator(outer-iterator<Const> i);
    […]
  };
}
```

11. Modify 26.7.17.3 [range.split.iterator], class `split_view::iterator` synopsis, as indicated:

```
namespace std::ranges {
  template<forward_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to>
  class split_view<V, Pattern>::iterator {
  private:
    […]

    constexpr iterator(split_view& parent, iterator_t<V> current, subrange<iterator_t<V>> next);     // exposition only

  public:
    […]

    iterator() = default;
    constexpr iterator(split_view& parent, iterator_t<V> current, subrange<iterator_t<V>> next);
    […]
  };
}
```

12. Modify 26.7.17.4 [range.split.sentinel], class `split_view::sentinel` synopsis, as indicated:

```
namespace std::ranges {
  template<forward_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to>
  struct split_view<V, Pattern>::sentinel {
  private:
    sentinel_t<V> end_ = sentinel_t<V>();               // exposition only

    constexpr explicit sentinel(split_view& parent);     // exposition only

  public:
    sentinel() = default;
    constexpr explicit sentinel(split_view& parent);
    […]
  };
}
```

13. Modify 26.7.22.3 [range.elements.iterator], class template `elements_view::iterator` synopsis, as indicated:

```
namespace std::ranges {
  template<input_range V, size_t N>
    requires view<V> && has-tuple-element<range_value_t<V>, N> &&
             has-tuple-element<remove_reference_t<range_reference_t<V>>, N> &&
             returnable-element<range_reference_t<V>, N>
  template<bool Const>
  class elements_view<V, N>::iterator {
  private:
    […]
```

```
    constexpr explicit iterator(iterator_t<Base> current);     // exposition only

  public:
    […]

    iterator() requires default_initializable<iterator_t<Base>> = default;
    constexpr explicit iterator(iterator_t<Base> current);
    […]
  };
}
```

14. Modify 26.7.22.4 [range.elements.sentinel], class template `elements_view::sentinel` synopsis, as indicated:

```
namespace std::ranges {
  template<input_range V, size_t N>
    requires view<V> && has-tuple-element<range_value_t<V>, N> &&
             has-tuple-element<remove_reference_t<range_reference_t<V>>, N> &&
             returnable-element<range_reference_t<V>, N>
  template<bool Const>
  class elements_view<V, N>::sentinel {
  private:
    […]

    constexpr explicit sentinel(sentinel_t<Base> end);     // exposition only

  public:
    sentinel() = default;
    constexpr explicit sentinel(sentinel_t<Base> end);
    […]
  };
}
```